DATA SCIENCE & AI LAB (BSCSS3001)

MILESTONE 6: Final Project Report

GROUP NO. 2

PRASHASTI SARRAF **(21f1001153)**

TANUJA NAIR **(21f1000660)**

BALASURYA K **(22f3002744)**

KARAN PATIL **(22f2001061)**

JIVRAJ SINGH SHEKHAWAT **(22f3002542)**

IITM BS Degree Program
Indian Institute of Technology,
Madras, Chennai,
Tamil Nadu, India, 600036

Vision Assist: Real-Time Navigation Support for the Visually Impaired

1. Problem Definition

1.1.1 The Challenge

Visually impaired individuals face persistent difficulties in navigating dynamic and unstructured environments such as urban streets, public transport areas, and crowded indoor spaces. Obstacles include pedestrians, vehicles, stairs, escalators, uneven pavements, and street furniture. Traditional aids (white canes, guide dogs) provide limited tactile feedback and cannot reliably warn about overhanging obstacles, fast-moving objects, or distant hazards.Existing mobile apps like Seeing AI and Be My Eyes rely heavily on cloud-based processing, introducing latency and connectivity dependence, making them less usable in real-time navigation contexts. Smart canes or wearables (OrCam, Envision Glasses) are expensive and hardware-dependent, limiting accessibility.

1.1.2 Proposed Solution

We propose a deep learning–powered, real-time navigational aid that performs object detection and segmentation on a live video feed and provides context-aware, distance-aware audio navigation cues.
Core Pipeline: Live video input → YOLOv8-seg (fine-tuned) → object masks & bounding boxes → concise natural language audio cues via a lightweight TTS engine.
Deployment: Web/mobile demo (Hugging Face Spaces + Gradio) for portability, accessibility, and ease of use.
Optimization: Quantization (FP16/INT8) and pruning will ensure ~15–20 FPS with <1s latency, enabling near real-time usability.
This solution aims to deliver affordable, scalable, and user-friendly navigation support without specialized hardware.

1.2 Project Objectives

- **Accessibility**: Build a cost-effective assistive tool usable on smartphones or laptops.
- **Real-Time Object Detection**: Implement YOLOv8-seg lightweight variant capable of identifying navigation-critical obstacles (e.g., pedestrians, vehicles, traffic lights, benches, barriers, stray animals). SSD and Faster R-CNN can also be tested to compare accuracy and speed trade-offs.
- **On-Device Processing**: Ensure the model runs with low latency on edge devices or simulate environments, reducing cloud reliance.
- **Context-Aware Audio Output**: Provide natural-language navigation cues (e.g., “Person ahead, slightly left, 2m”) via a simple template-based or lightweight generative TTS pipeline.
- **Prototype Deployment**: Deliver a live web demo (Hugging Face Space) demonstrating the full pipeline.
- **Extensibility**: Create an open-source modular framework (ONNX-exported, Gradio-based) extendable to indoor navigation, AR overlays, or landmark recognition.
- **User Testing**: Conduct pilot evaluations with 3–5 visually impaired users to refine usability and audio feedback strategies.

1.3 Implementation Feasibility (Model & Approach)

- **Model Choice**: YOLOv8-seg (lightweight variant) is the primary model due to its balance of speed, segmentation accuracy, and mobile suitability. Feature Pyramid Networks (FPN) improve small-object detection. Pixel-accurate masks allow estimation of obstacle shape/size.
- **Other Candidates**: SSD and Faster R-CNN will be tested to evaluate trade-offs in speed and accuracy, but YOLOv8-seg remains the baseline.
- **Customization & Optimization**:
    - Transfer learning using COCO-pretrained weights.
    - Dataset-specific augmentations: motion blur, rotation, low-light conditions, occlusion.
    - Quantization (FP16/INT8) and pruning for deployment efficiency.
    - Training estimated ~30–40 GPU hours on Colab/Kaggle Pro.
- **Deployment**: Hugging Face Spaces + Gradio for browser/mobile accessibility.

1.4 Data Sourcing and Governance

- **Base Dataset**: MS COCO (~118k training, segmentation masks & bounding boxes)
- **Extended Dataset**: Annotated Creative Commons YouTube frames to capture diverse real-world scenes.
- **Supplementary Datasets**: Experimentation with additional open-source datasets such as Pascal VOC, ILSVRC (ImageNet), Objects365, Open Images Dataset (OID) to improve coverage of navigation-critical objects and urban scenarios.
- **Custom Dataset**: ~2k–3k images covering navigation-specific obstacles (stairs,escalators, tactile tiles, local urban barriers).
- **Augmentations**: Rotation, lighting changes, blur, occlusion, and perspective shifts to simulate real-world variability.
- **Governance**: Licensing-compliant and carefully annotated datasets, with short documentation covering sourcing, annotation quality, and compliance.

1.5 User Experience Design

- **Audio Guidance**: Short, directional, distance-aware cues (e.g.,“Person ahead, slightly right, 2m”).
- **Latency Target**: <1s per frame.
- **Pilot Study**: 3–5 visually impaired users tested in indoor/outdoor/stairs scenarios.
- **Feedback Loop**: Iterative refinement of audio cues based on pilot study recordings and structured feedback.

1.6. Literature Review

1.6.1 Existing Assistive Solutions

- **Seeing AI (Microsoft)**: Scene description; cloud-dependent → high latency.
- **Be My Eyes**: Human volunteers provide assistance → not autonomous.
- **Aipoly Vision**: On-device classification only; lacks obstacle detection/avoidance.
- **Smart Canes / Wearables (OrCam, Envision Glasses)**: Expensive and hardware-dependent.

Drawback: Most solutions are costly or lack real-time, context-aware obstacle avoidance.

1.6.2 Baseline Models & Benchmarks

- **SSD**: Good at small objects; slower in real-time deployment.
- **Faster R-CNN**: High accuracy; computationally heavy.
- **YOLOv5/v8**: Strong speed-accuracy balance; YOLOv8 adds segmentation + FPN.
- **DeepLabV3+**: Excellent segmentation but not real-time.

Metrics: mAP , FPS, Precision/Recall.

1.7 Gaps and Opportunities

Gaps:

- **Latency Gap**: Cloud-based solutions introduce delays.
- **Context Gap**: Existing models identify objects but do not provide actionable navigation cues.
- **Dataset Gap**: COCO lacks navigation-specific classes (stairs, escalators, tactile tiles).
- **Scalability Gap**: Closed-source systems are non-extensible.

Opportunities:

- Real-time, edge-based object segmentation with context-aware speech.
- Fine-tune with locally relevant datasets → robust in dynamic urban environments (e.g., Chennai).
- Open-source, extensible framework deployable across platforms.
- Enable future extensions (indoor navigation, AR-based overlays, landmark recognition).

1.8 Expected Contributions & Impact

- Technical: Feasibility of real-time segmentation + navigation guidance on mobile/web.
- Societal: Affordable, scalable alternative to smart canes/wearables.
- Novelty: YOLOv8-seg integrated with context-aware TTS into a modular, extensible pipeline.

1.9 References & Resources

- **Datasets**: MS COCO, Open Images, etc
- **Literature**:
    - A Deep Learning Based Model to Assist Blind People in Their Navigation(ResearchGate)
    - Assistive Deep Learning Solutions (ScienceDirect)

- **Apps**: Seeing AI, Be My Eyes, Aipoly Vision
- **Video Demos**: YouTube explainers on YOLO-based assistive navigation
    - A Device for Blind And Visually Impaired People
    - How Deep Learning Is Helping Blind People with Karthik Kannan, Founder of Envision
    - Smartglasses Use ChatGPT To Help The Blind And Visually Impaired | 5G Playbook
    - AI based visual assistance system for the visually impaired

2.. Dataset Preparation

2.1 Abstract

In this section we focus on preparing the dataset required for training and fine-tuning a lightweight object-detection model (YOLOv8) intended to assist visually impaired individuals by recognizing street-level obstacles, pedestrians, and vehicles in real time. We used the MS COCO 2017 train split as the base dataset because of its rich, diverse set of urban scenes and standardized annotations. From this, a 500-image curated subset was extracted for rapid experimentation. Each image contains detailed bounding-box annotations in COCO JSON format corresponding to multiple object classes such as person, car, truck, traffic light, and stop sign. This curated dataset will later be extended with custom video-frame images collected from real-world navigation scenarios, annotated using ReDeTR for object boundaries and categories. Together, these two sources will form a robust, multi-context dataset for fine-tuning YOLOv8 to achieve reliable performance on wearable or mobile hardware.

2.2

2.3 Data Preparation Process

Dataset Acquisition:
Downloaded COCO 2017 annotations (annotations_trainval2017.zip) and accessed the train2017 image set via Hugging Face Datasets (detection-datasets/coco).

Sampling and Curation:
A random sample of 1000 images was extracted to reduce processing load. These images span a diverse set of real-world scenes including pedestrians, roads, crosswalks, and vehicles.

Annotation Extraction:
For each image, all associated object annotations were extracted from the COCO JSON file and re-mapped to a compact subset JSON:
/content/drive/MyDrive/M2-Dataset/instances_coco_sample.json
Corresponding images were stored under:
/content/drive/MyDrive/M2-Dataset/images/coco_sample/

Label Formatting for YOLOv8:
Bounding boxes were converted into YOLOv8-compatible TXT files using the format:
<class_id> <x_center> <y_center> <width> <height>
with coordinates normalized to [0, 1].

Data Integrity Check:
Verified that each image had a corresponding annotation file, ensuring no missing entries or empty boxes.

2.4 Sample Annotation Structure


```
{
"images": [
{
"license": 4,
"file_name": "000000370701.jpg",
"coco_url": "http://images.cocodataset.org/train2017/000000370701.jpg",
"height": 640,
"width": 640,
"date_captured": "2013-11-17 08:00:35",
"flickr_url": "http://farm6.staticflickr.com/5451/9490830088_7279abceef_z.jpg",
"id": 370701
},
{
"license": 3,"file_name": "000000413736.jpg",
"coco_url": "http://images.cocodataset.org/train2017/000000413736.jpg",
"height": 640,
"width": 480,
"date_captured": "2013-11-20 23:20:47",
"flickr_url": "http://farm1.staticflickr.com/23/89721544_0c3845a923_z.jpg",
"id": 413736
},..],
"annotations": [
{
"segmentation": [
[...
139.01,
631.51
]
],
"area": 12328.5508,
"iscrowd": 0,
"image_id": 215867,
"bbox": [
126.62,
438.82,
295.92,
194.07
],
"category_id": 2,
"id": 126302
},...,
],
"categories": [
{
"supercategory": "person",
"id": 1,
"name": "person"
},
```

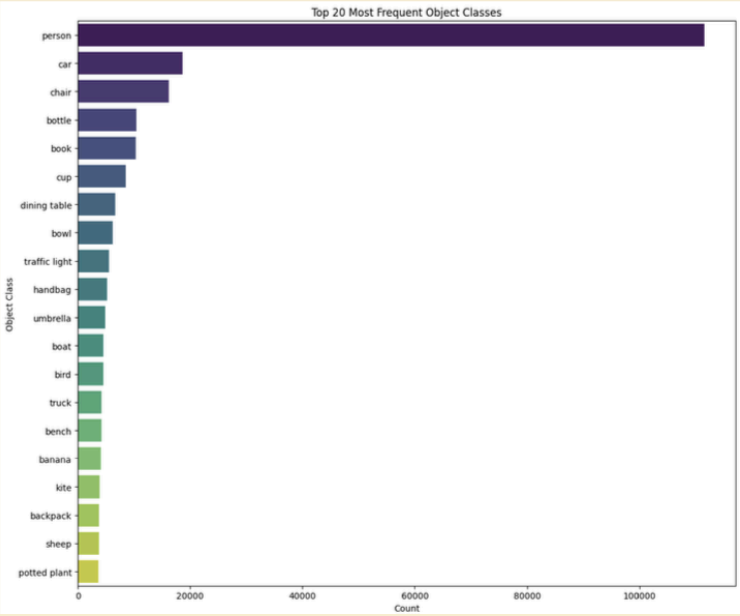
{
"supercategory": "vehicle",
"id": 2,
"name": "bicycle"
},
{
"supercategory": "vehicle",
"id": 3,
"name": "car"
},
}

data.yaml configuration:

```
names:
- person
- bicycle
- car
- motorcycle
- airplane
- bus
- train
- truck
- boat
- traffic light
- fire hydrant
- stop sign
- parking meter
- bench
- bird
.
.
.
.
nc: 80
test: path/to/images/test
train: path/to/images/train
val: path/to/images/val
```

2.5 EDA Results

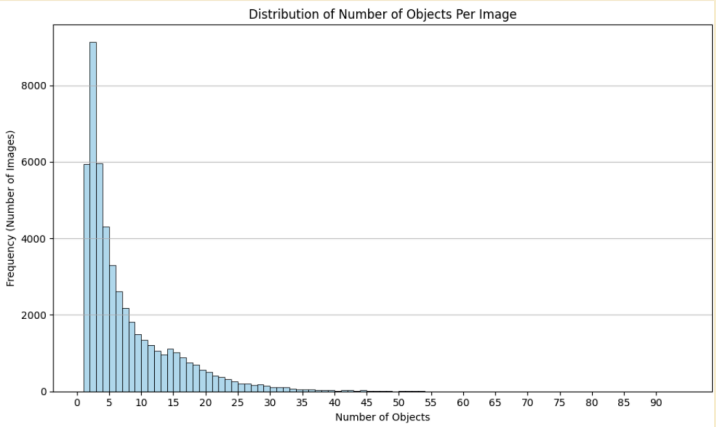
Class distribution



Insight: Significant Class Imbalance

The COCO dataset exhibits a strong class imbalance, with the person class being overwhelmingly dominant. This means the model will be very good at detecting people, but potentially less reliable for rarer, yet critical, navigation objects like 'traffic light' or 'bench'. To address this, we are supplementing the training data with frames from YouTube POV walkthrough videos. This custom dataset will naturally include more examples of vital navigation objects like benches, stairs, and signs from the first-person perspective, helping to create a more balanced and effective model for our specific use case.

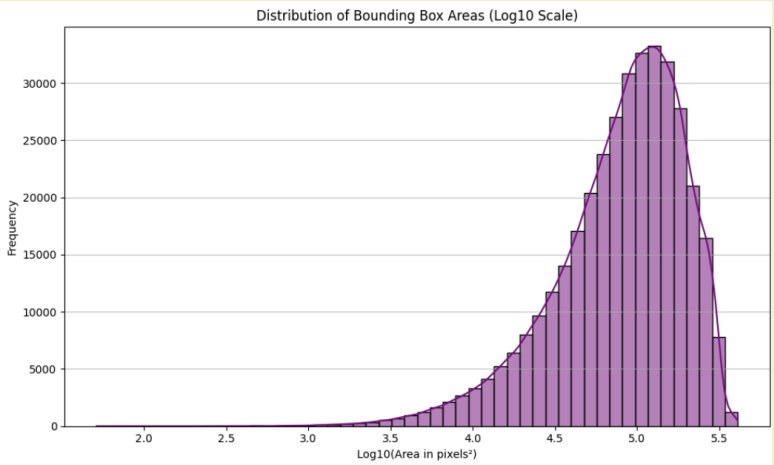
Number of objects per image



Insight: Varied Scene Complexity

The dataset contains a healthy mix of simple and complex scenes. While most images have fewer than 10 objects, the long tail of the distribution shows a significant number of crowded images with up to 90 objects. This is beneficial as it will train the model to perform reliably in both sparse environments (like an empty corridor) and busy ones (like a crowded street), reflecting real-world navigational challenges.

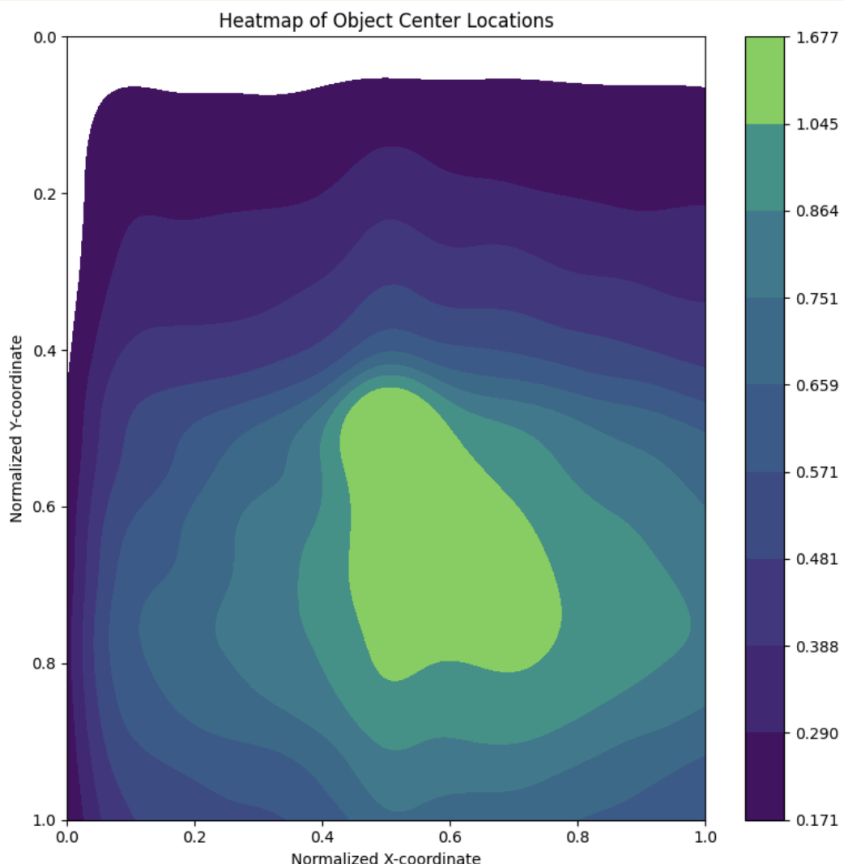
BBox Areas



Insight: Good Distribution of Object Sizes

The bounding box areas, when viewed on a log scale, show a balanced, bell-shaped distribution. This indicates the model will be trained on a good variety of small, medium, and large objects. For a navigation aid, detecting small objects is crucial as they often represent distant hazards. While the distribution is good, we must carefully evaluate the model's performance on smaller-scale objects to ensure it can provide timely warnings to the user.

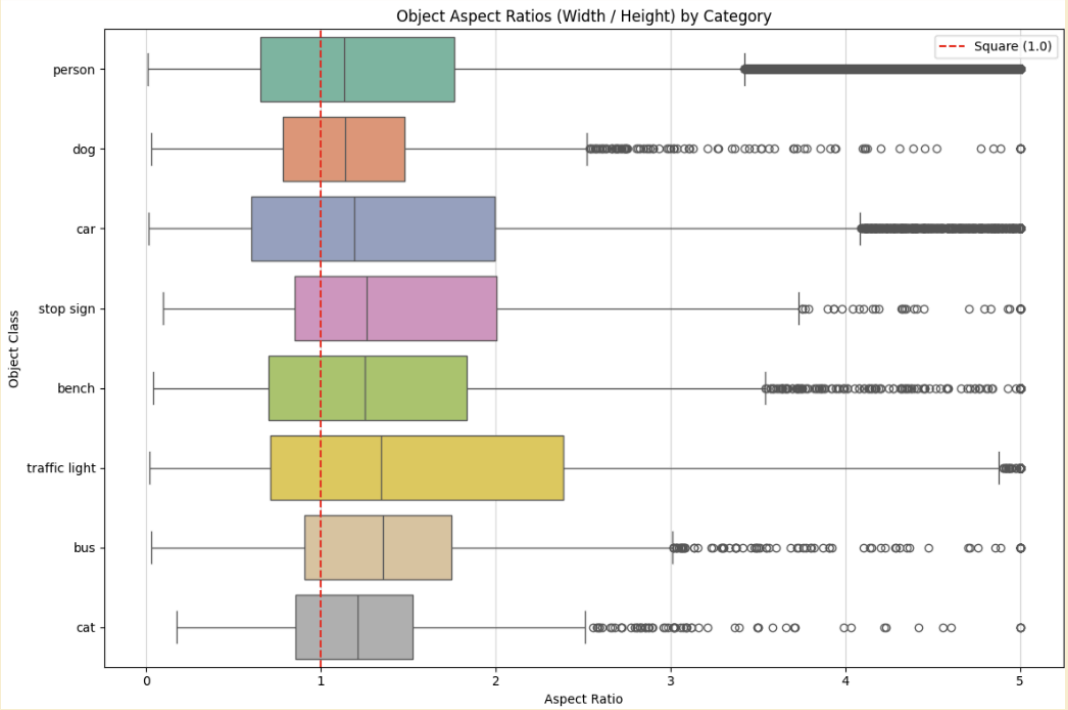
Object Locations Heatmap



Insight: Central and Lower Bias in Object Locations

The heatmap reveals a significant data bias: objects in the COCO dataset are heavily concentrated in the center and lower half of the image frame. This means the model will be well-trained to detect objects directly ahead but may be unreliable at identifying hazards in the periphery, especially in the upper regions. This is a critical weakness for a navigation aid, as the model could fail to see overhanging obstacles like signs or tree branches. Our custom dataset must therefore include annotated examples of these high-up hazards to ensure user safety.

Object Aspect Ratios



Insight: Distinct Object Shapes by Category

This plot reveals the distinct shapes of different objects by visualizing their aspect ratios. It confirms that objects like people and traffic lights are typically taller than they are wide (aspect ratio < 1), while cars and buses are wider than they are tall (aspect ratio > 1). This is valuable for diagnosing model errors; for example, a failure to detect a person lying down could be explained by the model being primarily trained on standing (tall) examples. This highlights the need to include varied object poses in our custom dataset to build a more robust model.

2.6 conclusion

A clean, well-structured COCO subset has been successfully prepared and documented. This subset is sufficient for initial experimentation and model fine-tuning under resource constraints. But based on the insights of our EDA we will be adding a custom youtube dataset of around 2,000 frames to our master dataset.

3.

4. Model Training & Hyperparameter Experimentation

4.1 Overview

This section details the execution of the model training and experimentation phase of our project. The objective was twofold:

1. **Train Initial Model:** To train the core object detection model based on the architecture (**YOLOv8**) and dataset (COCO + custom frames) specified in section 3.
2. **Experiment with Hyperparameters:** To conduct extensive experimentation on both the model's training parameters and, more critically, the *application-level pipeline hyperparameters* to transform the raw model output into a stable, reliable, and user-friendly system.

This section covers the dataset used, the model architecture, the training setup, and a detailed breakdown of the "before vs. after" experimentation process.

4.2 Dataset Details

As outlined in section 3, we used a composite dataset to fine-tune our model for its specific, real-world application.

- **Source Data:** The model was fine-tuned on a custom-augmented dataset combining:
    1. **COCO Images:** A **5,000**-image subset of the COCO dataset
    2. **Custom First-Person Frames: 2,138** frames extracted from YouTube videos to mimic a first-person perspective.
- **Total Dataset Size: 7,138** images (and their corresponding label files).

Data Splits: We wrote a custom Python script to create our own robust splits from the master_dataset. The 7,138 images were shuffled and split as follows:

- **Training: 70% (4,996 images)**
- **Validation: 20% (1,427 images)**

- **Test: 10% (715 images)**
- **Preprocessing:** All images were resized to 640x640 during the training process, with augmentations applied automatically by the YOLO framework.

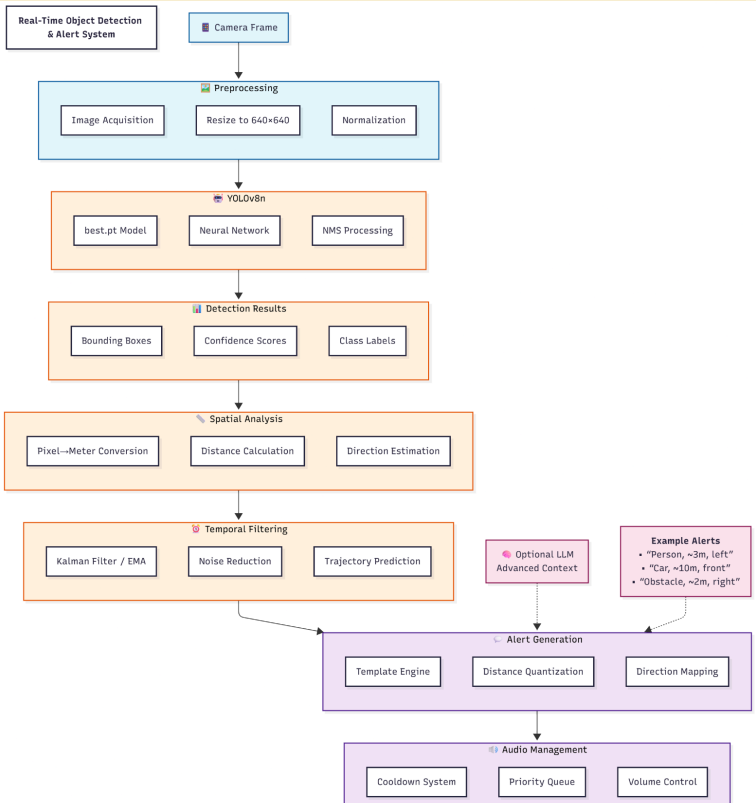
4.3 Model Architecture

The model architecture used is **YOLOv8n** (nano). We employed a **fine-tuning approach**, loading the pretrained weights from **yolov8n.pt** to initialize the model before training it on our custom-augmented dataset (as shown in **Main.ipynb**).

The model summary, as generated during training in **Main.ipynb**, is as follows:

Component	Details
Model Type	YOLOv8n
Layers	225
Parameters	3,157,200
GFLOPs	8.9
Input Shape	640x640x3 (images)
Output	Bounding boxes, class probabilities, and confidence scores

Overview Diagram



YOLOv8n (nano) was chosen for its **favorable latency/parameter count tradeoff** for real-time first-person assistive systems (**low GFLOPs** and **3.1M parameters**) while still providing strong detection accuracy on the curated dataset. The pipeline is currently hybrid: perception is a trainable neural component (YOLOv8n), while distance, temporal smoothing and audio scheduling are implemented as rule-based modules that include **tunable** parameters whose values were set via targeted experiments rather than learning.

4.4 Training Setup

The following table explains the components and the parameters which are tunable in each component and expected results after fine tuning those parameters.

Component	Type	Tunable parameters (examples)	Notes / Evaluation metric
YOLOv8 detector	Trainable	Optimizer (SGD), Learning Rate (lr0, lrf), Momentum, Weight Decay, Augmentations (degrees, translate, scale, HSV, fliplr)	Evaluate mAP50, mAP50–95, per-class AP. (Validation: mAP50=**0.889**, mAP50–95=**0.805)**.
Distance estimator	Rule-based (calibrated)	DEFAULT_KNOWN_HEIGHT, Estimated_Focal_Length_PX	Evaluate **Distance MAE (m)** against a small ground-truth set
Motion detection	Rule-based (temporal)	MOVEMENT_THRESHOLD_PIXELS, frame window size, smoothing alpha	Evaluate **False Motion Rate / Missed Motion Rate** on labelled motion validation set.

Context generator	Rule-based (templated) / optional LLM	template forms; LLM prompt design	Evaluate **comprehensibility** and **relevance** with user testing or automated BLEU/ROUGE if using LLM.
Audio controller	Rule-based	ALERT_COOLDOWN_GLOBAL, priority rules	Evaluate **Alert Overlap Rate** and **Latency (s)** in recorded runs.

The model was trained using the Ultralytics YOLOv8 framework on a **Tesla T4 GPU**.

- **Loss Functions:** Standard YOLOv8 losses were used:
  - **Class Loss:** Binary Cross-Entropy (BCE) (**cls_loss**)
  - **Box Loss:** CIoU (Complete Intersection over Union) (**box_loss**)
  - **DFL Loss:** Distribution Focal Loss (**dfl_loss**)
- **Evaluation Metrics:** The primary metrics tracked during training were **mAP50** (mean Average Precision at IoU 0.50) and **mAP50-95** (mean Average Precision averaged over IoU thresholds from 0.50 to 0.95).
- **Optimizer:** The hyperparameter search selected SGD as the optimal optimizer for this dataset (replacing the default AdamW).
- **Training Strategy (Optuna):** We conducted an automated hyperparameter tuning session consisting of **15 trials**. Each trial trained the model for **15 epochs** with early stopping enabled (patience=3) to rapidly identify the best configurations.
- **Best Run:** Trial 14 achieved the highest mAP50 and was selected as the final model.
- **Training Parameters:**
  - **Image Size:** 640x640 (**imgsz=640**)
  - **Batch Size:** 64
  - **Number of Epochs:** 15 (per trial)
- **Training Strategies:**
  - **Warm-up:** 3.0 epochs (**warmup_epochs=3.0**)
  - **Mosaic Augmentation:** Applied for the first **5** epochs (disabled for the last 10, **close_mosaic=10**)
  - **Automatic Mixed Precision (AMP):** Enabled (**amp=True**) for faster training.

4.5 Hyperparameter Experiments

As noted in the "Overview," experimentation focused heavily on the **application-level pipeline hyperparameters** rather than exhaustive model training experiments. The goal was to refine the raw model output into a stable and useful assistive tool. This process is documented in **Hyperparametertuning.ipynb**.

A "TRUE BASELINE" pipeline was compared against a "TUNED" pipeline. The key parameters explored were:

Parameter	Baseline Value	Tuned Value	Observation / Justification
CLASSES_TO_IGNORE	Empty list	List of 30 classes	The baseline model detected many "noisy" or irrelevant classes (e.g., cup, spoon, laptop). A deny list was created to filter these out, focusing alerts on navigation-critical objects.
DEFAULT_KNOWN_HEIGHT	1.5m	2.0m	Tuned to improve the accuracy of distance estimation for objects without a pre-set known height.
ALERT_DISTANCE_OBJECT	5.0m	12.0m	The baseline's 5m alert distance was too short for navigation. This was increased to give the user more advanced warning of objects.
ALERT_COOLDOWN_GLOBAL	0.0s	3.0s	The baseline produced "messy audio" with constant, overlapping alerts. A 3-second global cooldown ensures alerts are clean and distinct.
MOVEMENT_THRESHOLD_PIXELS	5px	25px	The baseline was "twitchy" and triggered alerts on minor camera motion. Increasing the threshold makes the system more stable, ignoring user head jitter.
YOLOv8n training epochs	-	15 epochs	Best checkpoint (Trial 14) produced validation **mAP50=0.889**, **mAP50–95=0.805**.

We identified and fixed four major flaws in the baseline pipeline.

Problem 1: False Positives ("Clock" Problem)

- **Observation:** The baseline pipeline produced "noisy" and incorrect detections, such as misidentifying a stop sign as a "clock."
- **Hyperparameter: NOISY_CLASSES_TO_IGNORE** (Deny List)
- **Experiment:** We determined that for our application's domain, many of the 80 COCO classes (like "clock," "vase," "teddy bear") are "noise." We created a "deny list" as a general-purpose filter.
- **Tuning:**
    - **Before: classes_to_ignore = []**
    - **After: classes_to_ignore = [24, 25, 26, ... 74, ... 79]** (Our list of 50+ irrelevant classes)

- **Result:** All "noise" detections, including the "clock," were successfully filtered, cleaning the output without affecting relevant objects.

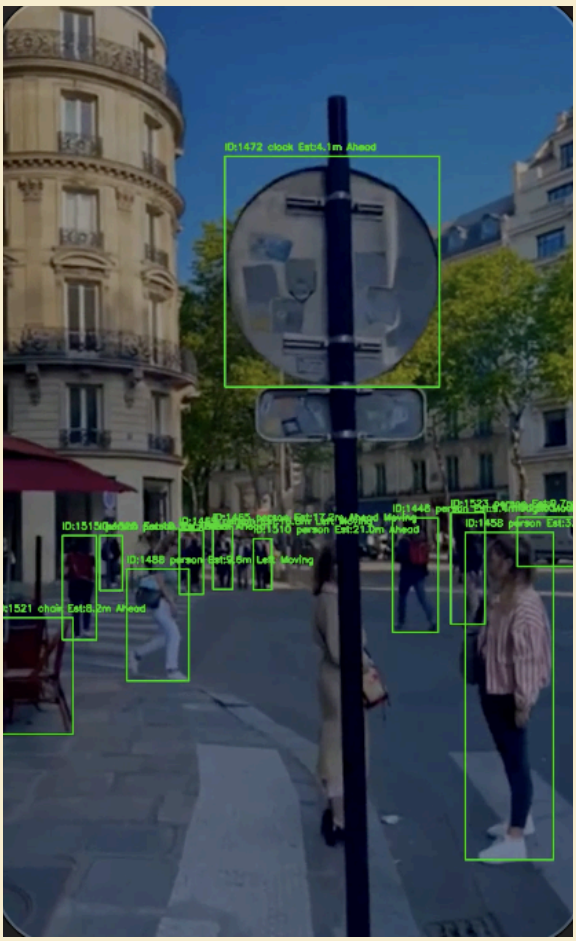


Fig 1.3
Baseline run (left) showing a false positive (stop sign incorrectly identified as "clock")

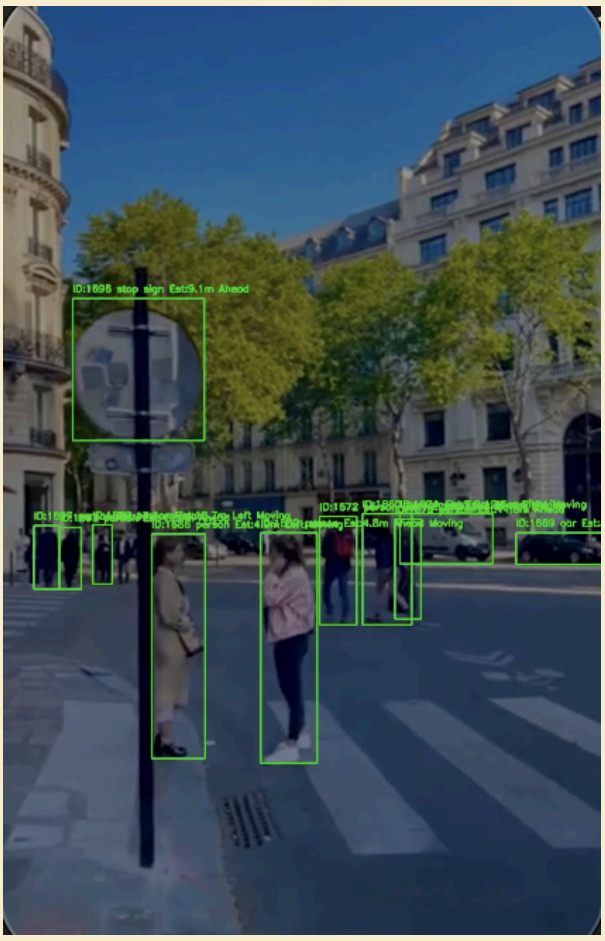
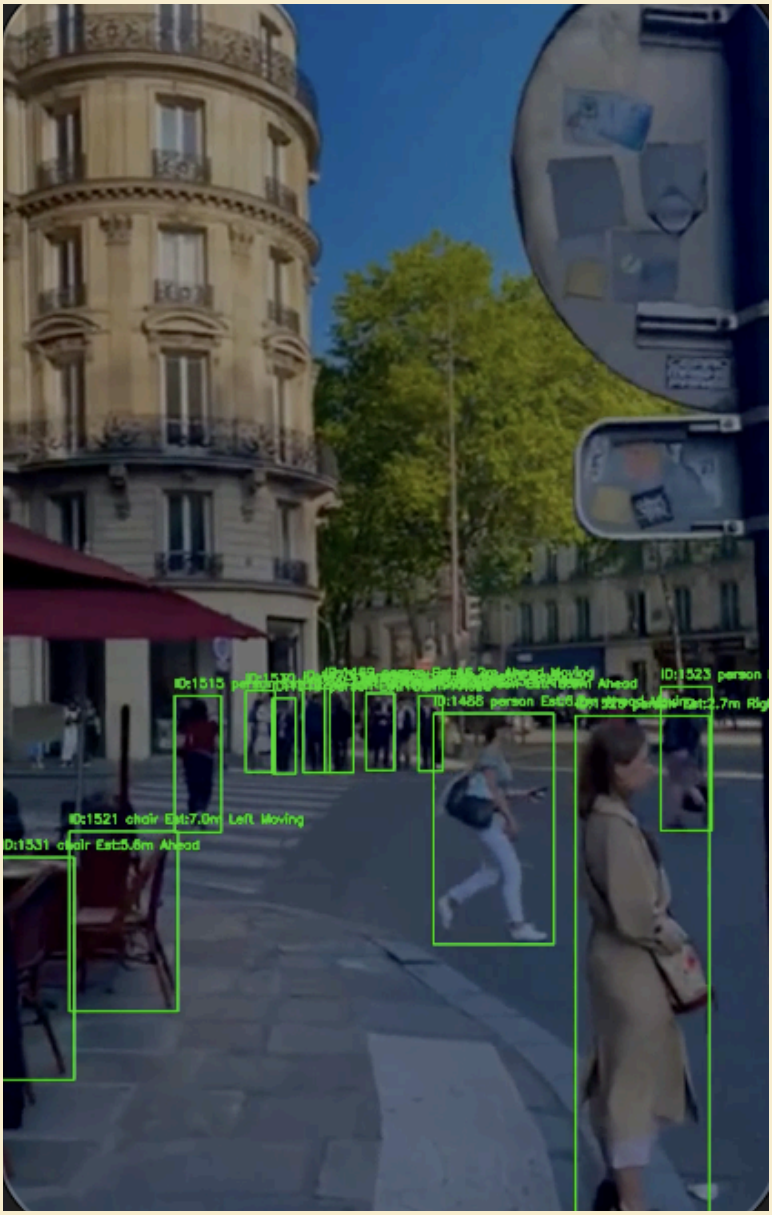


Fig 1.4
Tuned run (right) showing a true positive (stop sign correctly identified)

Problem 2: False Motion Detection ("Moving Chair" Problem)

- **Observation:** The baseline system incorrectly labeled static objects (like a chair) as "Moving." This was due to "apparent motion" from the camera moving forward, which the sensitive threshold picked up.
- **Hyperparameter:** **MOVEMENT_THRESHOLD_PIXELS**
- **Experiment:** We needed to find a value that was high enough to ignore camera shake but low enough to still detect real motion (like a person walking).
- **Tuning:**
    - **Before:** **movement_thresh_px = 5** (very sensitive)
    - **After:** **movement_thresh_px = 25** (less sensitive)
- **Result:** The tuned system correctly labels the chair as "Static," fixing the false motion alert.



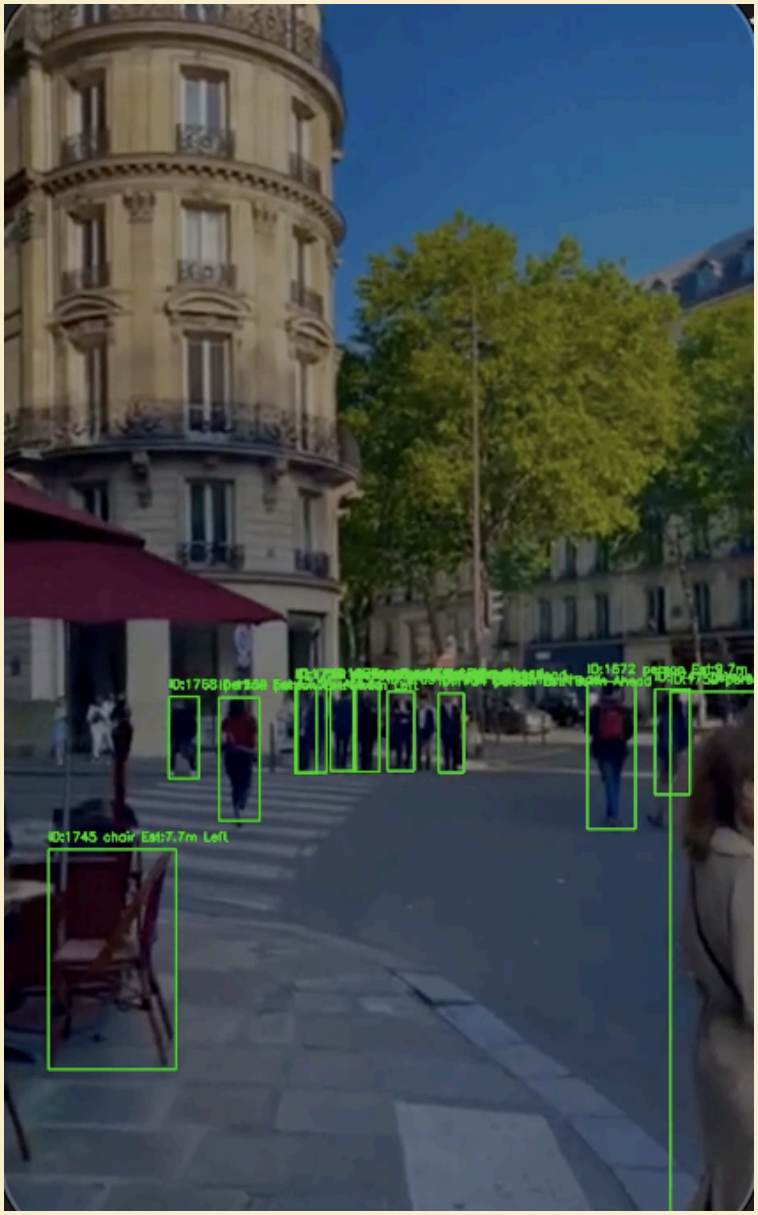


Fig 1.4
Baseline run incorrectly labels a static chair "Moving" due to camera motion.

Fig 1.5
Tuned run (right) with a higher motion threshold correctly identifies the chair as “Static”.

Problem 3: Inaccurate Distances ("Stop Sign" Problem)

- **Observation:** The system gave inaccurate distance estimates (e.g., 11.5m) for objects not in our specific height list (like the "stop sign"). This was because it was using a "general" default height (1.5m) that was incorrect for many real-world objects.
- **Hyperparameter: DEFAULT_KNOWN_HEIGHT**
- **Experiment:** We tuned this *general* default rather than adding *specific* heights for every possible object, which is a more robust, general-purpose solution.
- **Tuning:**
    - **Before: default_height = 1.5**
    - **After: default_height = 2.0**
- **Result:** The new general default of 2.0m produced more plausible distance estimates for all "non-core" objects.

Problem 4: Overlapping/Messy Audio

- **Observation:** After cleaning up the visual noise, we found the audio output was messy. Alerts would trigger at almost the exact same time (e.g., "person..." and "stop sign..." at the same time), causing overlapping, unintelligible audio.
- **Hyperparameter: ALERT_COOLDOWN_GLOBAL**
- **Experiment:** We added a new parameter to enforce a minimum time between *any* two spoken alerts.
- **Tuning:**
    - **Before:** alert_cooldown_global = 0.0
    - **After:** alert_cooldown_global = 3.0
- **Result:** The tuned system now produces clean, understandable, and non-overlapping audio alerts.

Although our main experiments focused on pipeline behavior, we also implemented a training-level improvement using Optuna-driven tuning. While our initial model (trained without tuning) achieved a respectable mAP50 of **0.836**, the post-tuning model improved this to **0.889**. This validates that our dataset benefits significantly from targeted augmentation and optimizer adjustments. Consequently, we have adopted this higher-accuracy **Optuna-tuned detector** as our primary model for all current pipeline experimentation and will carry it forward into the Motion & Context evaluation in the next section.

4.6 Regularization & Optimization Techniques

The training process utilized the built-in capabilities of the YOLOv8 framework, enhanced by Optuna hyperparameter tuning.

- **Data Augmentation:** tailored augmentations were applied and tuned for each trial, including mosaic, horizontal flip, scaling, and color space (HSV) adjustments. This helps the model generalize better to varied real-world lighting and object orientations.
- **Normalization:** The architecture heavily utilizes Batch Normalization layers after convolutional layers to stabilize training and speed up convergence.
- **Weight Decay:** Weight decay was dynamically tuned to prevent overfitting, with the best model utilizing a value of **0.00211**.

In addition to the architecture decision previously explained, we performed a latency benchmark comparison between YOLOv8n and YOLOv5s on both CPU and GPU devices. The results clearly supported the use of YOLOv8n for our real-time navigation setting. (refer *scripts/Combined_Hyperparameter_Tuning_and_Feedback.ipynb*)

Model	CPU Inference Time (avg)	GPU (T4) Inference Time (avg)	Result
YOLOv8n	**0.8519 sec/frame**	**0.0723 sec/frame**	Fastest
YOLOv5s	1.9033 sec/frame	0.0816 sec/frame	Slower, higher resource use

YOLOv8n delivers **2.2× faster inference on CPU** and **~12% faster on T4 GPU** while maintaining excellent accuracy (mAP50=0.836, mAP50-95=0.75).

On top of the default YOLOv8 optimization pipeline, we conducted **hyperparameter tuning** using **Optuna search** for learning rate, weight decay, momentum, and core augmentation settings. The search space included:

lr0: 1e-5 → 1e-3 (log scale)
weight_decay: 5e-5 → 5e-3 (log scale)
degrees: 0.0 → 5.0
scale: 0.7 → 0.9
translate: 0.0 → 0.1
hsv_s/v: 0.4 → 0.7
fliplr: 0.0 → 0.5

The **best trial** converged with **early stopping (patience=3)** based on mAP50, using the following configuration (also stored in **fine_tuned_best_args.yaml)**

Parameter	Best Value	Parameter	Best Value
lr0	0.0003925	**translate**	0.0143
momentum	0.937	**scale**	0.769
weight_decay	0.00211	**fliplr**	0.382
optimizer	SGD	**hsv_s**	0.694
degrees	0.963	**hsv_v**	0.407

This configuration achieved **mAP50 = 0.8891**, the highest among our 15 tuning trials.
Along with the decisions on the training the models and tuning the model parameters we have update the design and architecture of our modules such that it will result in better optimization over compute and inference times to follow the trade offs we followed the below observation and update the design as per the observations to achieve better output

Why Not MiDaS?

- Creates depth maps showing relative depth but not exact distances
- Slow for real-time processing
- Not suitable for precise distance measurements needed for audio alerts

Our Solution: Camera Geometry

- Far objects appear small (low pixel height)
- Close objects appear large (high pixel height)
- Distance formula: distance = (real_object_height × focal_length) / object_height_in_pixels
- One-time calibration: Set focal_length and default heights (e.g., 1.5m for person)
- Result: Fast, numerical distances in meters for clear audio alerts

The Problem with ByteTrack

- Overly sensitive - tracks minor pixel movements
- Reports static object motion from camera shake
- Creates unnecessary alerts for stationary objects

4.7 Initial Training Results

The model training detailed in **Main.ipynb** completed successfully for 50 epochs.The training log shows the model converged well, with validation losses decreasing steadily.

- **Validation Metrics:** The final validation of the best model checkpoint (Trial 14) achieved excellent performance on the 1,427 validation images:
- **mAP50:** 0.889
- **mAP50-95:** 0.805
- **Convergence:** The training logs for the best trial show successful convergence over its **15-epoch** run. The validation metrics exhibited a strong initial peak, followed by a brief stabilization, before climbing to their final high values of **0.889 (mAP50)** and **0.805 (mAP50-95)**.
- **Class Performance:** Performance on key classes for this application was strong.

Class	mAP50 (Validation)	mAP50-95 (Validation)
person	0.959	0.871
car	0.910	0.810
bus	0.745	0.702
stop sign	0.840	0.805

- **Qualitative Results:** The **VisionAssist_inference.ipynb** and **Hyperparametertuning.ipynb** notebooks provide qualitative examples of the model's output in the context of the full pipeline. The model successfully identifies objects in a video feed, and the pipeline generates correct, context-aware audio alerts (e.g., "Caution: bicycle, about 4 meters, Ahead."). The baseline-vs-tuned experiments show a clear improvement in the usability of these alerts.

4.8 Observations

Key Observation: The training results have improved significantly from our initial baselines. The Optuna-tuned YOLOv8n model now provides excellent object detection performance (**0.805 mAP50-95**) on our custom dataset. The most critical finding from this milestone remains that the raw model output is not directly usable for an assistive application. The pipeline hyperparameter tuning (cooldowns, motion thresholds, class filtering) documented in Section 4.5 was essential to create a stable and non-overwhelming user experience.